

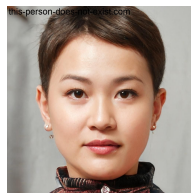
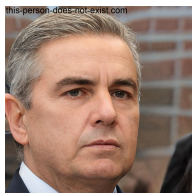
GANSpace: Discovering Interpretable GAN Controls

Erik Härkönen, Aaron Hertzmann, Jaakko Lehtinen, Sylvain Paris

Presented by Charumathi Badrinath, Eric Shen, and Skyler Wu

Motivation + Example

- GANs are models trained to generate realistic-looking images well—and they do!
- They have **not** been designed, however, to make the image-generation process itself interpretable, or to support edits to generated images
- E.g.: This GAN generates faces well and can take in high-level styles or criteria (gender, age, race), but is not designed to facilitate post-hoc *continuous* and *meaningful* edits, such as head angle, hair length, lighting...



Main Contributions and Key Ideas

1. Applying a simple technique (**PCA**) in the **latent space** of GANs gives semantically significant directions.
2. Perturbing inputs in these directions in certain layers of the GANs → interpretable edits in properties of generated images.
3. Such edits are high-level and nuanced (e.g. object shape to background landscape)
4. A user need only label each “control direction” once to edit the image

Novel setting:

unsupervised identification of **interpretable** directions in **existing** GANs

In other words, the authors posit that exploring the “**EiGAN**space” gives us new controls to edit and possibly better understand GANs at low cost

Main Contributions and Key Ideas



Perturbing the 15th principal component direction in the latent space, in the 8th layer.



Combining edits in various principal component directions to edit a generated photo.

GANs: Generative Adversarial Networks

Seminal Paper: Goodfellow et al. (2014)

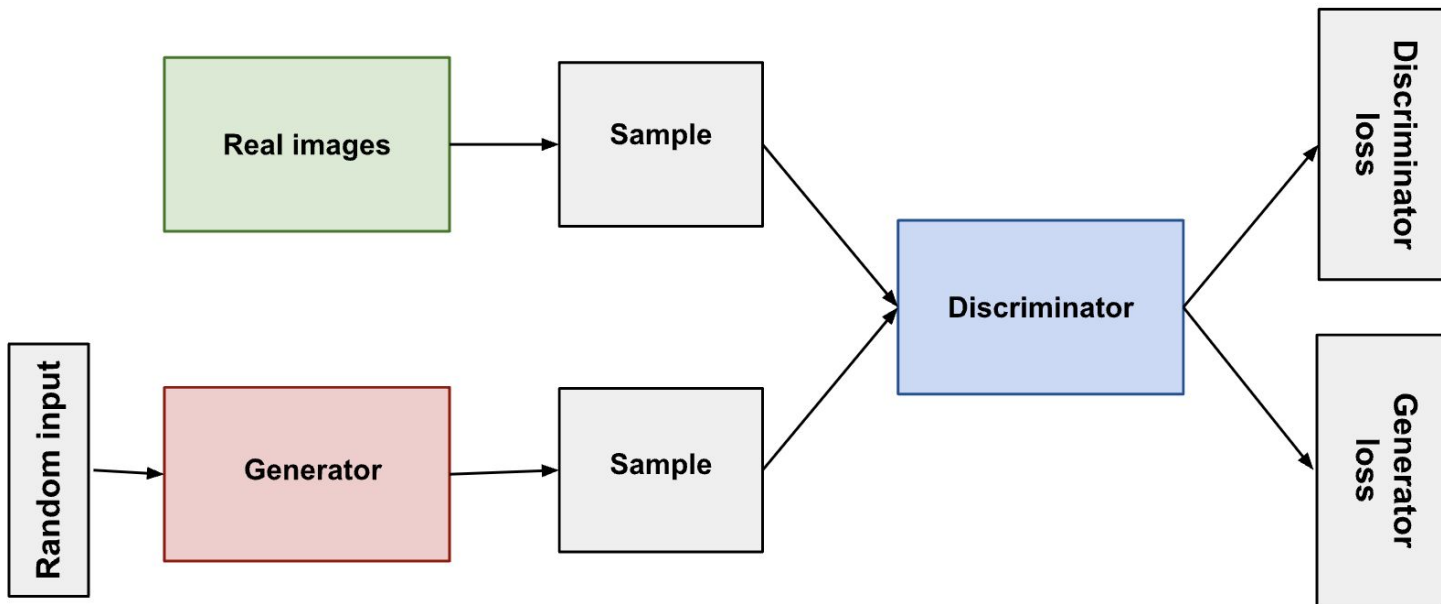
- Objective: **Generative Image Modeling**—create realistic-looking, artificial images that resemble the training data.
- GAN = Generator (G) + Discriminator (D) networks, both CNNs in image context.
 1. Generator: generate artificial images starting with noise input.

$$\underbrace{\mathbf{z} \sim p(\mathbf{z})}_{\text{Random Latent Vector}} \longrightarrow \underbrace{I = G(\mathbf{z})}_{\text{Artificial Image Generation (RGB)}} = \overbrace{(G_L \circ G_{L-1} \cdots \circ G_1)(\mathbf{z})}^{\text{Iterative Feature Outputs}} \underbrace{\hspace{10em}}_{\text{Feature Tensor, i.e. a Set of Feature Maps}}$$

2. Discriminator: try to distinguish generator's fake images from real training data.

GANs: Generative Adversarial Networks

Key Idea: “Back-and-Forth Between Generator and Discriminator”



BigGAN: Brock et al. (ICLR 2019)

1. Class-Conditional Image Generation
 - Given $\mathbf{z}$ from latent space and some class information (e.g., “dog”), generate the synthetic image.
 - BigGAN takes in class information as input $\rightarrow$ model *conditional distribution*!
2. Architecture designed for higher-resolution images and more parameters
3. “Skip-Z Inputs” in intermediate layers: $\mathbf{z}$ is imputed as a shared embedding into each layer

$$\underbrace{y_i}_{\text{Feature Output at Layer } i} = G_i(\underbrace{y_{i-1}}_{\text{Feature Output at Layer } i-1}, \underbrace{z}_{\text{Original Input}})$$

BigGAN: Brock et al. (ICLR 2019)

BigGAN → High Quality, High Resolution Images → Difficult to Interpret.



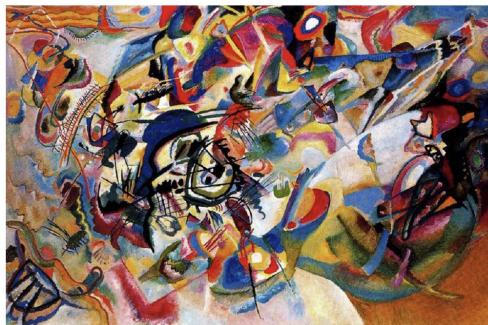
Class-Conditional Samples from BigGAN

StyleGAN: Karras et al. (CVPR 2019)

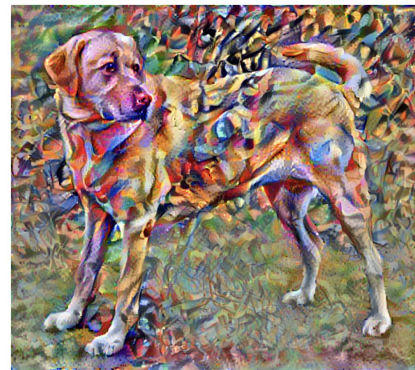
1. Inspiration: “**style transfer**” (content image + reference style reference image → blend into output image that keeps main aspects of content image, but appears to be “painted” in the “style” of the style reference image.
2. Motivation: want to better “control the image synthesis process”
 - Learned constant input + adjust “style” at each convolutional layer (e.g., pose, lighting, viewpoint)



+



=



StyleGAN: Karras et al. (CVPR 2019)

Relevant Mathematical Details:

1. No random input $\mathbf{z}$, rather—a fixed, constant learned input $\mathbf{y}_0$.
2. Slightly different intermediate layers and connections:

$$\mathbf{y}_i = G_i(\mathbf{y}_{i-1}, \mathbf{w}), \text{ with } \mathbf{w} = M(\mathbf{z}).$$

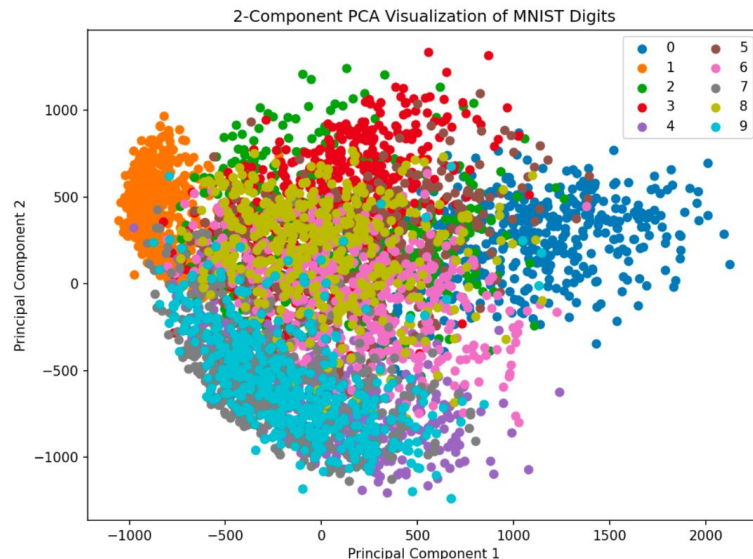
The $\mathbf{w}$ is a
non-linear
transformation
of the original
input $\mathbf{z}$.

M is a 8-Layer
Feed-Forward NN.
Potentially different
at each layer of
Generator → “style
mixing”

PCA: Principal Component Analysis

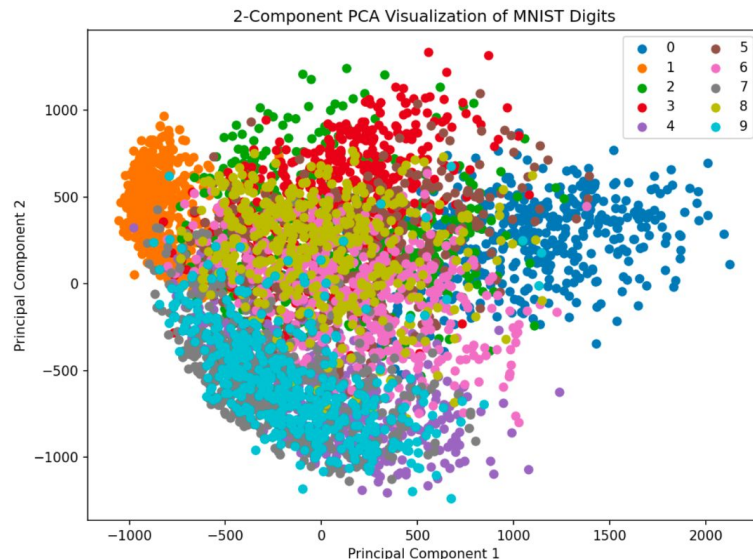
1. What is it?

- An unsupervised learning technique for finding a lower-dimensional representation of a dataset
- Find axes that explain the most variance in the data
- **Principal Components** are these axes: they equal the eigenvectors corresponding to the largest eigenvalues of the covariance matrix



PCA: Principal Component Analysis

1. What is it?
 - An unsupervised learning technique for finding a lower-dimensional representation of a dataset
 - Find axes that explain the most variance in the data
 - **Principal Components** are these axes: they equal the eigenvectors corresponding to the largest eigenvalues of the covariance matrix
2. Why is it useful in our context?
 - Each PCA basis vector helps better separate our data, or, in this context, latent space representation vectors



Related Work and Their Limitations

1. Trying to Learn Latent Directions via **Supervised Learning** (Jahanian et al., Goetschalckx et al., etc.)
 - Helpful in the sense of giving users more control over generated images
 - However, this is very expensive—need manual supervision of learned controls. Requires labeling training images, very costly

Related Work and Their Limitations

1. Trying to Learn Latent Directions via **Supervised Learning** (Jahanian et al., Goetschalckx et al., etc.)
 - Helpful in the sense of giving users more control over generated images
 - However, this is very expensive—need manual supervision of learned controls. Requires labeling training images, very costly
2. Training GANs with **inherently disentangled representations** (Ramesh et al.)
 - “Entanglement” = one model attribute, but multiple possible effects, either simultaneously or selectively
 - If successful, would permit fine-tuned edits on generated images
 - But, extremely, extremely computationally expensive

Related Work and Their Limitations

1. Trying to Learn Latent Directions via **Supervised Learning** (Jahanian et al., Goetschalckx et al., etc.)
 - Helpful in the sense of giving users more control over generated images
 - However, this is very expensive—need manual supervision of learned controls. Requires labeling training images, very costly
2. Training GANs with **inherently disentangled representations** (Ramesh et al.)
 - “Entanglement” = one model attribute, but multiple possible effects, either simultaneously or selectively
 - If successful, would permit fine-tuned edits on generated images
 - But, extremely, extremely computationally expensive

So, why not just take an already-trained GAN and analyze its latent / feature space?

Finding Useful Directions in z-space

- Latent space controls the image → we want to find directions in latent space that correspond to interpolating over a certain feature in pixel-space
- For example, can we find a direction corresponding to hair color?

Idea: can use PCA to find most influential directions

Issue: What to run PCA on?

- z-space prior? too simple to be informative
- pixel space? too complex to be informative

Finding Useful Directions in StyleGAN

StyleGAN maps $\mathbf{z} \rightarrow \mathbf{w}$ which controls “styling” at each layer...

Idea: Finding principal axes of distribution $p(\mathbf{w})$ could be useful!

1. Sample N random vectors $\mathbf{z}_{1:N}$, compute $\mathbf{w}_i = M(\mathbf{z}_i)$
2. Apply PCA to $\mathbf{w}_{1:N} \Rightarrow$ yields new basis $\mathbf{V}$ for $\mathbf{w}$ -space

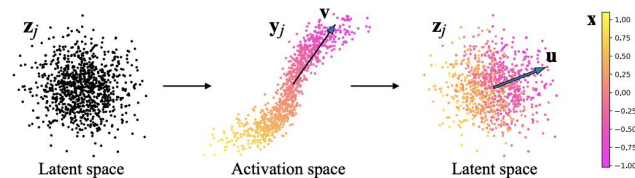
Recall that there is a deterministic mapping between latent vector and image $\Rightarrow$ given an image *defined by* $\mathbf{w}$ we can edit it by modifying $\mathbf{w}$ as follows:

$$\mathbf{w}' = \mathbf{w} + \mathbf{V}\mathbf{x}$$



controls magnitude of change
in each direction of the basis

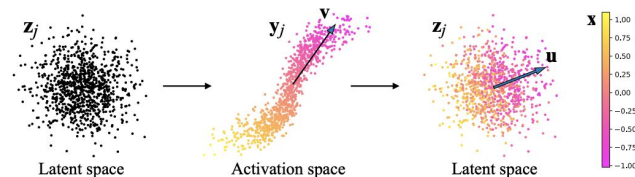
Finding Useful Directions in BigGAN



Issue: there is no styling parameter w ; instead, z is passed as additional input

Idea: 2-step approach

Finding Useful Directions in BigGAN



Issue: there is no styling parameter $\mathbf{w}$; instead, $\mathbf{z}$ is passed as additional input

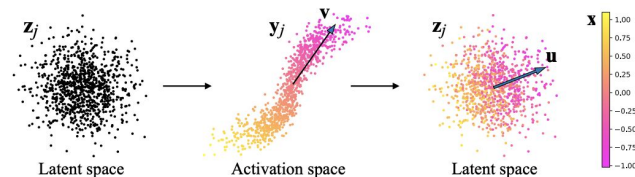
Idea: 2-step approach

(Step 1) Perform PCA at intermediate network layer i

1. Sample N random vectors $\mathbf{z}_{1:N}$
2. Process through model to produce N feature tensors at i th layer $\mathbf{y}_{1:N}$
3. Apply PCA to $\mathbf{y}_{1:N} \Rightarrow$ yields new basis $\mathbf{V}$ for $\mathbf{y}$ -space, data mean $\boldsymbol{\mu}$
4. Project $\mathbf{y}_{1:N}$ into new basis yielding vectors $\mathbf{x}_{1:N} = \mathbf{V}^T(\mathbf{y}_{1:N} - \boldsymbol{\mu})$

done at the first linear layer

Finding Useful Directions in BigGAN



Issue: there is no styling parameter $\mathbf{w}$; instead, $\mathbf{z}$ is passed as additional input

Idea: 2-step approach

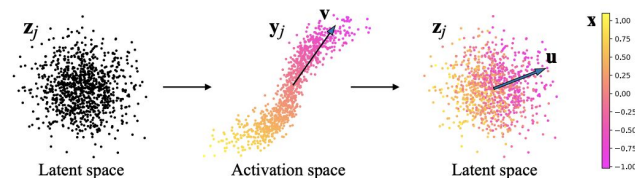
(Step 1) Perform PCA at intermediate network layer i

1. Sample N random vectors $\mathbf{z}_{1:N}$
2. Process through model to produce N feature tensors at i th layer $\mathbf{y}_{1:N}$
3. Apply PCA to $\mathbf{y}_{1:N} \Rightarrow$ yields new basis $\mathbf{V}$ for $\mathbf{y}$ -space, data mean $\boldsymbol{\mu}$
4. Project $\mathbf{y}_{1:N}$ into new basis yielding vectors $\mathbf{x}_{1:N} = \mathbf{V}^T(\mathbf{y}_{1:N} - \boldsymbol{\mu})$

(Step 2) Find latent directions corresponding to principal components

1. $\mathbf{U} = \operatorname{argmin} \sum_j \|\mathbf{U}\mathbf{x}_j - \mathbf{z}_j\|^2 \rightarrow$ “principal direction” matrix

Finding Useful Directions in BigGAN



Issue: there is no styling parameter $\mathbf{w}$; instead, $\mathbf{z}$ is passed as additional input

Idea: 2-step approach

(Step 1) Perform PCA at intermediate network layer i

1. Sample N random vectors $\mathbf{z}_{1:N}$
2. Process through model to produce N feature tensors at i th layer $\mathbf{y}_{1:N}$
3. Apply PCA to $\mathbf{y}_{1:N} \rightarrow$ yields new basis $\mathbf{V}$ for $\mathbf{y}$ -space, data mean $\boldsymbol{\mu}$
4. Project $\mathbf{y}_{1:N}$ into new basis yielding vectors $\mathbf{x}_{1:N} = \mathbf{V}^T(\mathbf{y}_{1:N} - \boldsymbol{\mu})$

(Step 2) Find latent directions corresponding to principal components

1. $\mathbf{U} = \operatorname{argmin} \sum_j \|\mathbf{U}\mathbf{x}_j - \mathbf{z}_j\|^2 \rightarrow$ “principal direction” matrix

We can edit an image defined by $\mathbf{z}$ by modifying $\mathbf{z}$ as follows

$$\mathbf{z}' = \mathbf{z} + \mathbf{U}\mathbf{x}$$

Granularity of Edits

Idea: Edits can be applied to all layers or just a few for more localized effects

- StyleGAN: pass the modified $\mathbf{w}'$ to some layers, pass the original $\mathbf{w}$ to others
- BigGAN: pass the modified $\mathbf{z}'$ to some layers and the original $\mathbf{z}$ to others + keep the $\mathbf{z}$ at the beginning of the network the same

Idea: Edits can be *composed* by moving in several principal directions simultaneously

Edit directions can be labelled as corresponding to a concept by users

Findings and Results

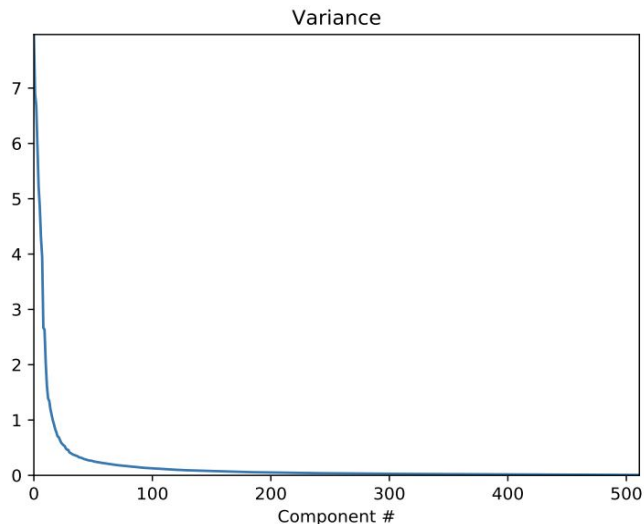


(a) Fix first 8 PCA coord., randomize remaining 504
(Pose and camera fixed, appearance changes)

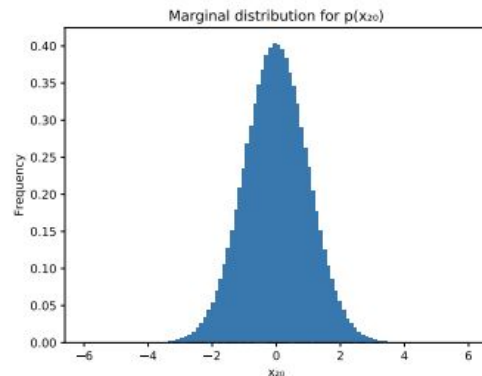
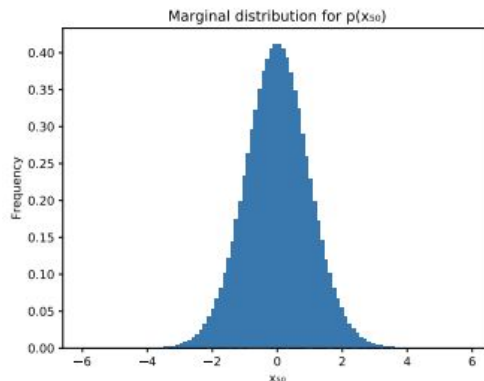
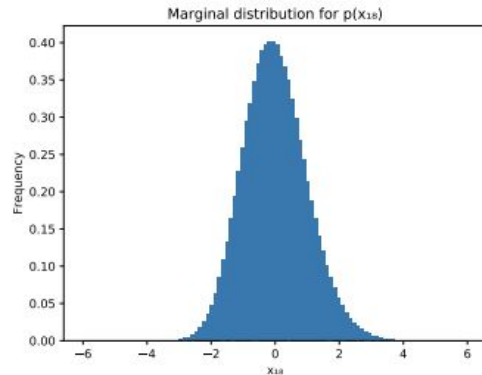
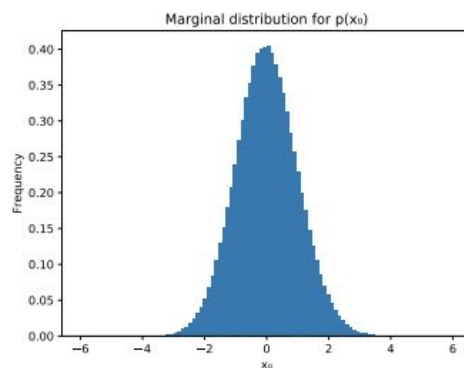
(b) Randomize first 8 PCA coord., fix remaining 504
(Appearance fixed, pose changes)

Changing the **first 20** principal components (PCs) in latent space correspond to changes in image **layout**, **configuration**, and **perspective**; later PCs dictate object appearance, background, and smaller details.

Findings and Results



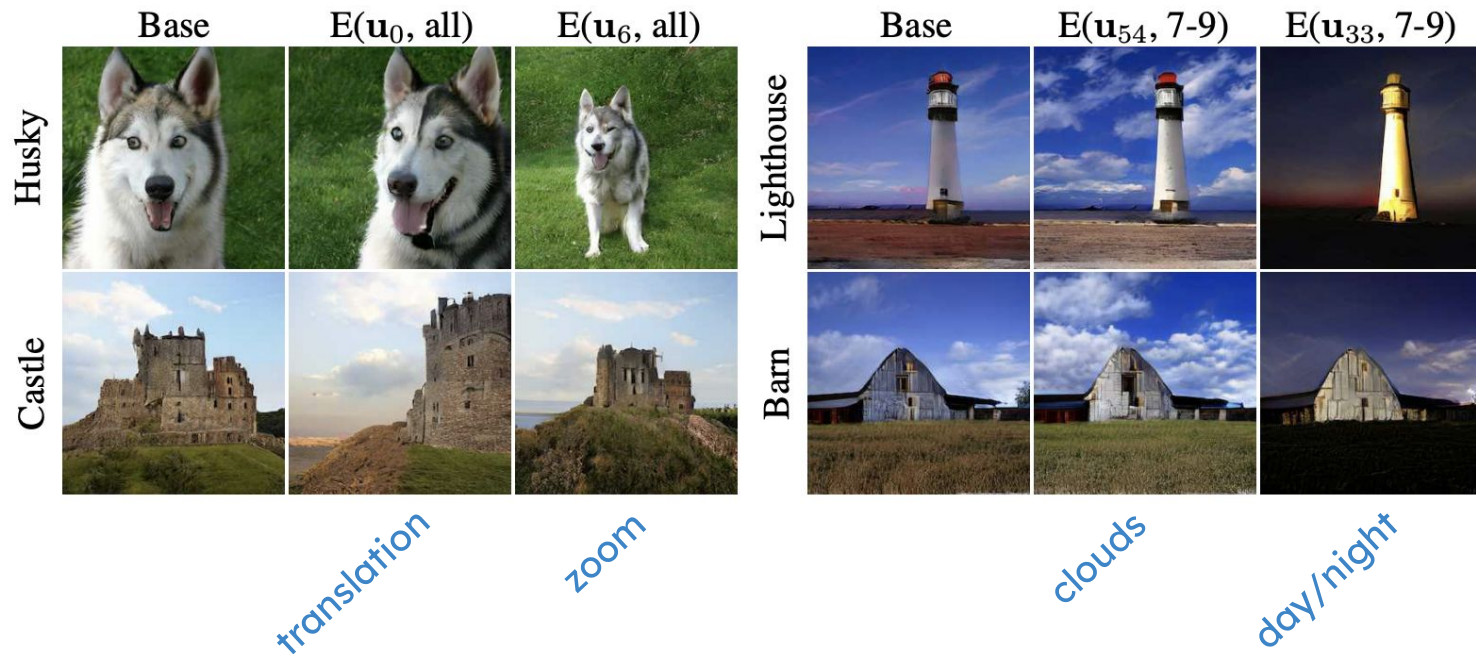
Variance in latent space is tied to magnitude of image change



Empirical PDFs of Principal Components are bell curves

StyleGAN's **first 100 PCs** sufficiently explain overall image appearance, and are distributed unimodally and almost independently

Findings and Results



BigGAN's PCs seem to be **class-independent**, i.e. PCA gives the same results for different images in different classes

Analyzing GANs Through Principal Components

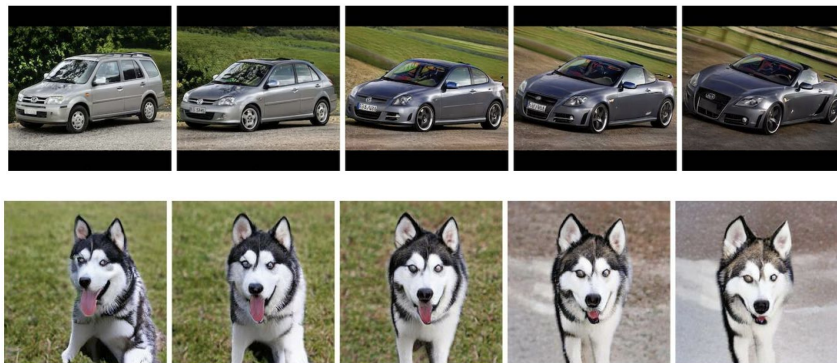
Limitations in edit expressivity through PCs seem to be inherited by dataset limitations:

- StyleGAN faces cannot be translated in the image
- PCs corresponding to wrinkles/makeup have no effect on children/men



Entanglement/Superposition of different attributes is displayed by some PCs:

- One PCs for StyleGAN cars corresponds to sportiness and more open-road backgrounds
- Rotating a dog causes its mouth to open



Comparison with Related Techniques

- Perturbing the latent space in **randomly-chosen directions** results in virtually indiscernible changes in mixtures of several attributes
- By contrast, PCA gives a more separated, ordered series of stylistic edit axes
- Results are similar to those achieved through supervised methods, while being simpler to compute (no extra training)
- PCs provide many more edit directions that would be costly to individually discover via supervised techniques
- More entanglement



(d) StyleGAN1 FFHQ Smile $E(\mathbf{v}_{44}, 3)$



Strengths/Weaknesses

A cursory list:

- Method is unsupervised and computationally easier to implement than supervised methods

Strengths/Weaknesses

A cursory list:

- Method is unsupervised and computationally easier to implement than supervised methods
- Analysis of principal component directions elucidates possible interpretations of GANs' latent space representations

Strengths/Weaknesses

A cursory list:

- Method is unsupervised and computationally easier to implement than supervised methods
- Analysis of principal component directions elucidates possible interpretations of GANs' latent space representations
- Gives users flexibility: after labelling directions, users can perform numerous edits on various styles of the image

Strengths/Weaknesses

A cursory list:

- Method is unsupervised and computationally easier to implement than supervised methods
- Analysis of principal component directions elucidates possible interpretations of GANs' latent space representations
- Gives users flexibility: after labelling directions, users can perform numerous edits on various styles of the image
- Approach seems to be limited to the specific GAN architecture

Strengths/Weaknesses

A cursory list:

- Method is unsupervised and computationally easier to implement than supervised methods
- Analysis of principal component directions elucidates possible interpretations of GANs' latent space representations
- Gives users flexibility: after labelling directions, users can perform numerous edits on various styles of the image
- Approach seems to be limited to the specific GAN architecture
- Principal components *a priori* are not linked with specific stylistic meaning: users would have to try changing values manually to see their effects

Strengths/Weaknesses

A cursory list:

- Method is unsupervised and computationally easier to implement than supervised methods
- Analysis of principal component directions elucidates possible interpretations of GANs' latent space representations
- Gives users flexibility: after labelling directions, users can perform numerous edits on various styles of the image
- Approach seems to be limited to the specific GAN architecture
- Principal components *a priori* are not linked with specific stylistic meaning: users would have to try changing values manually to see their effects
- Choosing which principal components to modify at what layers seems arbitrary

Strengths/Weaknesses

A cursory list:

- Method is unsupervised and computationally easier to implement than supervised methods
- Analysis of principal component directions elucidates possible interpretations of GANs' latent space representations
- Gives users flexibility: after labelling directions, users can perform numerous edits on various styles of the image
- Approach seems to be limited to the specific GAN architecture
- Principal components *a priori* are not linked with specific stylistic meaning: users would have to try changing values manually to see their effects
- Choosing which principal components to modify at what layers seems arbitrary
- Entanglement of principal component styles hinders practicality

Strengths/Weaknesses

A cursory list:

- Method is unsupervised and computationally easier to implement than supervised methods
- Analysis of principal component directions elucidates possible interpretations of GANs' latent space representations
- Gives users flexibility: after labelling directions, users can perform numerous edits on various styles of the image
- Approach seems to be limited to the specific GAN architecture
- Principal components *a priori* are not linked with specific stylistic meaning: users would have to try changing values manually to see their effects
- Choosing which principal components to modify at what layers seems arbitrary
- Entanglement of principal component styles hinders practicality
- Does this constitute actual interpretability for the GANs?

Questions for the Audience

- Are these principal components a useful tool for interpreting GANs?
- How does their utility for interpretability compare with other methods we have studied? Do you think this method truly satisfies interpretability?
- Finding: most conceptually-aligned PCs for StyleGAN in w-space, most useful PCs for BigGAN originated in the first linear layer (then, projected to z-space).
- ^Do you think these findings provide novel insights into the models?
- How could this tool help identify and reduce biases in a model?

Findings and Results

Experiment targets: **StyleGAN**, **BigGAN** (n.b. methods differed for the models)

Main Finding Ideas:

1. Changing the **first 20** principal components (PCs) in latent space correspond to changes in image **layout**, **configuration**, and **perspective**; later PCs dictate object appearance, background, and smaller details
2. StyleGAN's **first 100 PCs sufficiently explain overall image** appearance, and are distributed unimodally and almost independently
3. BigGAN's PCs seem to be **class-independent**, i.e. PCA gives the same results for different images in different classes

Interpreting the Latent Space of GANs for Semantic Face Editing

Authors: Yujun Shen, Jinjin Gu, Xiaoou Tang, Bolei Zhou

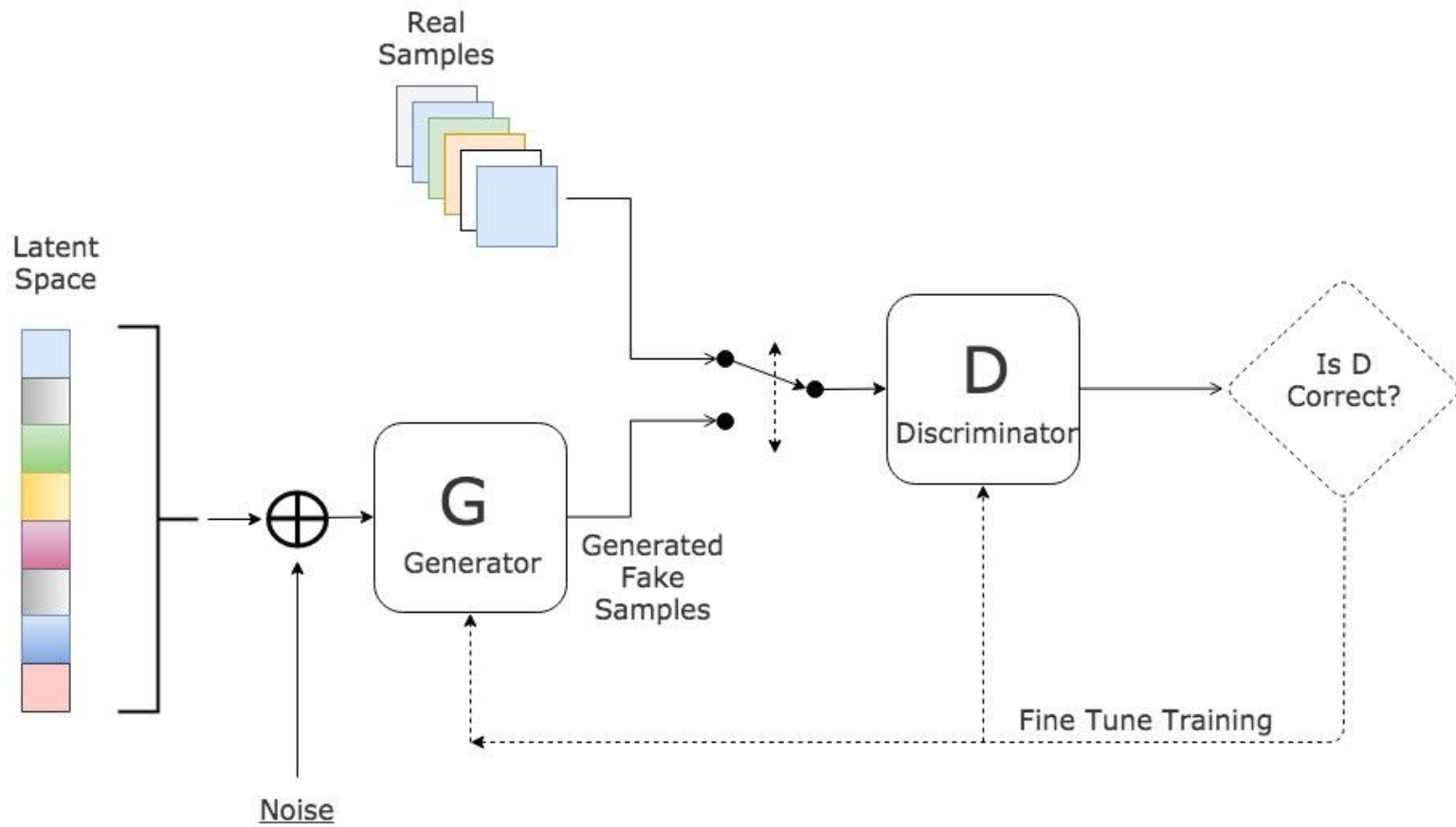
Presented by Anat Kleiman, Gustaf Ahdritz, Cynthia Chen, and Luke Bailey

Presentation Roadmap

- Introduction
- Method
- Experiments

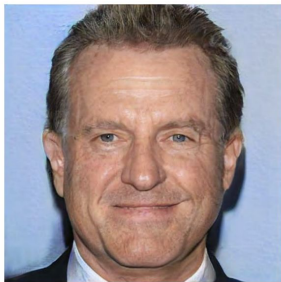
Background: What are GANs?

- **Generative Adversarial Network (GAN):** generative models that generate incredibly realistic synthetic data that mimic underlying training data
 - **Generator:** tries to generate samples similar to training data
 - **Discriminator:** tries to distinguish between generated samples (fake) and training data (real)
- **Latent representation:** a low-dimensional representation of an image (eg: a 5-dimensional vector)
- **Latent space:** the set of all possible latent representations
 - Through adversarial training, the generator learns a mapping (“latent distribution”) from points in the latent space to real images



Background: GANs for Image Editing

- **Latent code:** the version of an image in the latent space
 - GAN inversion: identifying the latent code for a given image such that the generator could construct the original image
- **Semantic editing:** editing the latent code to manipulate some features of the resulting image, such that only the desired features are changed



Related Work

- Exploring latent spaces in GANs
 - Laine et al: exploring making the output smoothly vary from one synthesis to another in the latent space
 - Radford et al observe the *vector arithmetic property*
 - Can be thought of as an additive property in the embedding space, e.g.:
$$\text{vector}(\text{"King"}) - \text{vector}(\text{"Man"}) + \text{vector}(\text{"Woman"}) = \text{vector}(\text{"Queen"})$$
 - Applications in camera motion (Jahanian et al.), scene synthesis (Yang et al.), memorability (Goetschalckx et al.)
- Semantic face editing with GANs
 - Previous methods involved carefully designed loss functions or training new models with specialized architectures
 - No existing method to perform controlled facial editing using GANs by varying latent codes

Research Questions

- **How do semantics in the latent space originate, and how are they organized?**
 - Does there exist structure in the latent space relating to disentangled semantic representations?
- **What does a GAN actually learn with respect to the latent space?**
 - How does the GAN connect the latent space and the image semantic space?
- **How can the latent code be used for image editing?**
 - How are various semantic attributes of an individual's face (gender, age) determined and entangled with each other?

Contributions and Main Findings

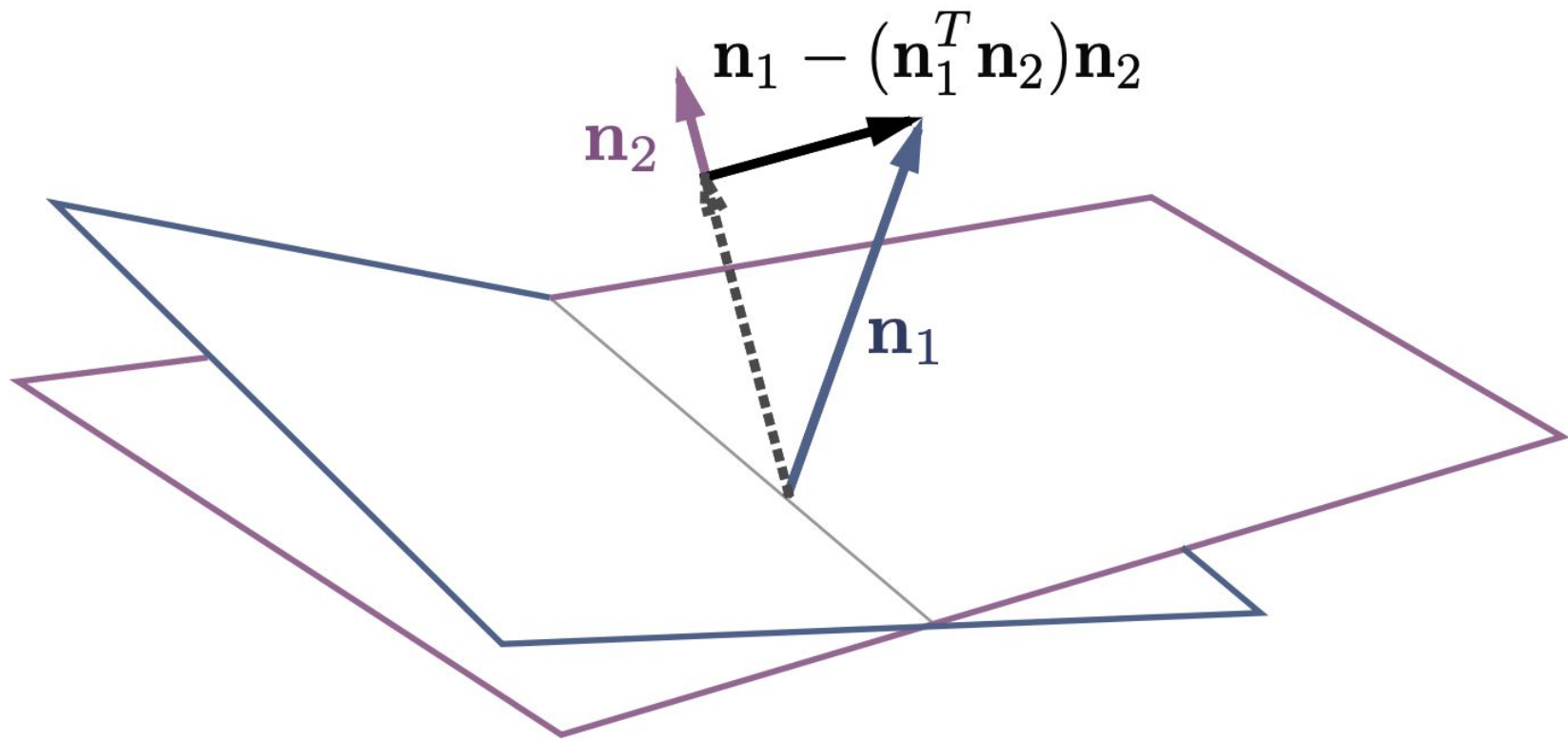
- Proposed **InterFaceGAN**, a framework to identify the semantics encoded in the latent space of face synthesis models
 - Observed that **GANs learn latent subspaces corresponding to specific attributes**
- Showed that InterFaceGAN **enables semantic face editing with any pre-trained GAN**
 - Found theoretical and experimental results to verify that linear subspaces align with semantics emerging in the latent space
- Applied InterFaceGAN to **real image editing**
 - Successfully edited and manipulated the attributes of real faces by varying the latent code

Presentation Roadmap

- Introduction
- **Method**
- Experiments

$$\mathbf{d}(\mathbf{n}, \mathbf{z}) = \mathbf{n}^T \mathbf{z}.$$

$$f(g(\mathbf{z})) = \lambda d(\mathbf{n}, \mathbf{z})$$

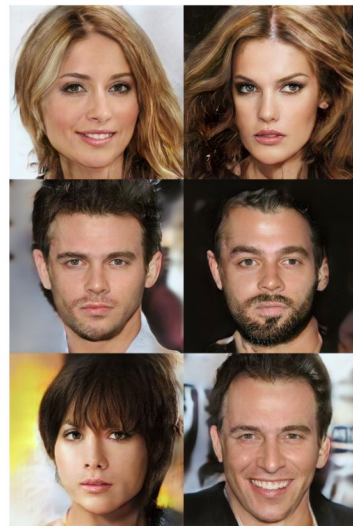
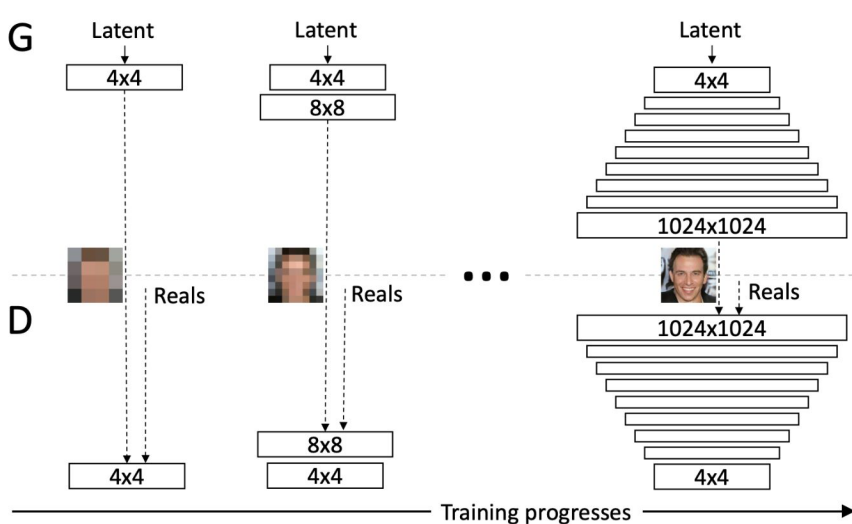


Presentation Roadmap

- Introduction
- Method
- Experiments

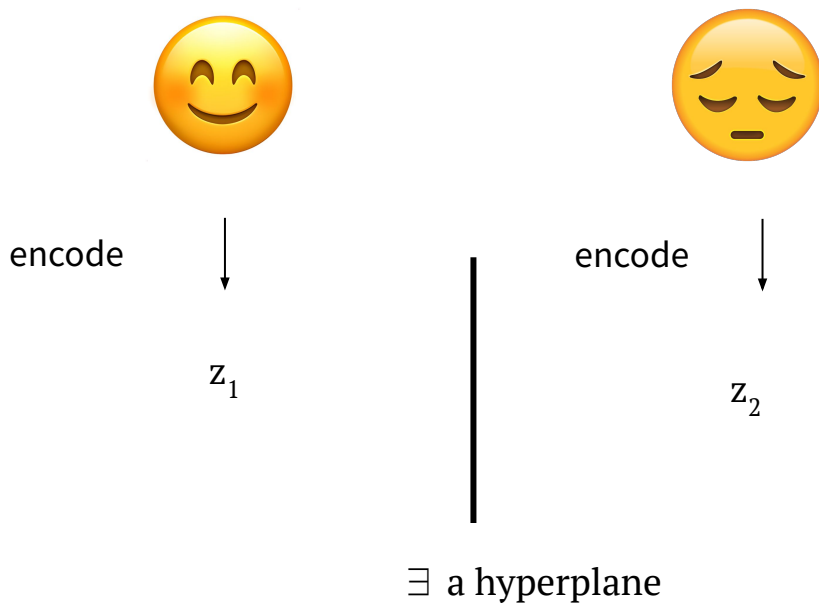
Experiment 1: Latent Space Separation

- **Model** - PGGAN (for experiment 2 and 3 also)
 - trained by growing both the generator and discriminator progressively: starting from a low resolution, add new layers that model increasingly fine details as training progresses.
 - Trained on CelebA-HQ face attributes dataset (each face is labeled with a subset of 40 attribute annotations). 30,000 images in total.



Experiment 1: Latent Space Separation

The framework is based on an assumption that for any binary attribute, there exists a hyperplane in latent space such that all samples from the same side have the same attribute.



Experiment 1: Latent Space Separation

Train five independent linear SVMs on pose, smile, age, gender, eyeglasses, latent codes and then evaluate them on the validation set (6K samples with high confidence level on attribute scores) as well as the entire set (480K random samples).

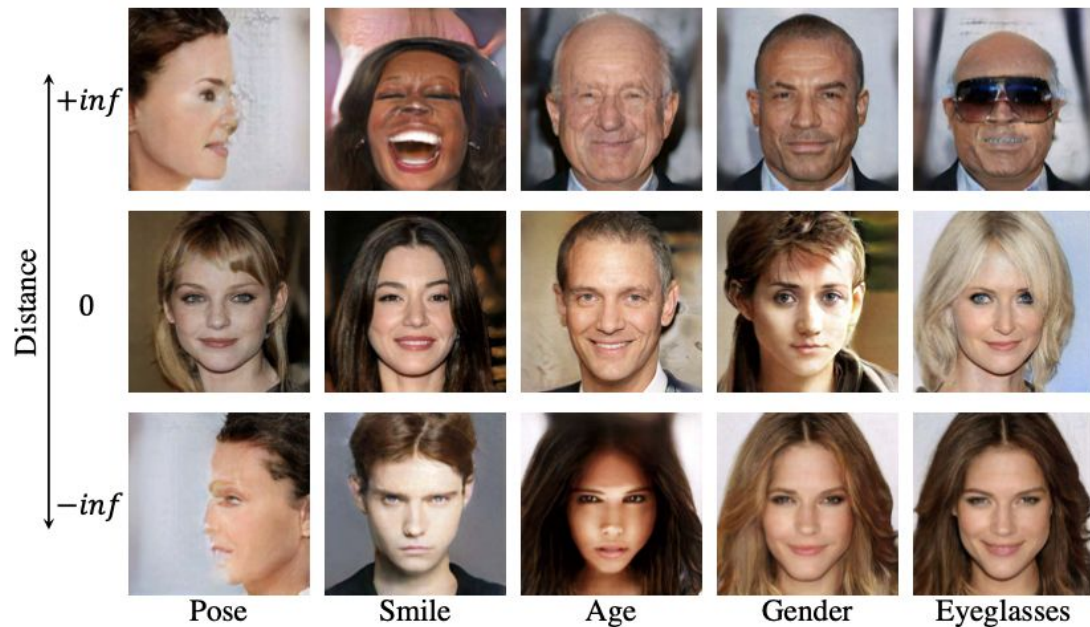
Datasets created by generating from GAN and classifying with ResNet-50 trained on the CelebA-HQ dataset.

Table 1: Classification accuracy (%) on separation boundaries in latent space with respect to different attributes.

Dataset	Pose	Smile	Age	Gender	Eyeglasses
Validation	100.0	96.9	97.9	98.7	95.6
All	90.3	78.5	75.3	84.2	80.1

Experiment 1: Latent Space Separation

Choose a z far away from and on decision boundary and generate from it



Experiment 2: Latent Space Manipulation

Verify whether the semantics found by InterFaceGAN are manipulable

Single attribute

Generate central image then move away from boundary

Works in both positive and negative directions.

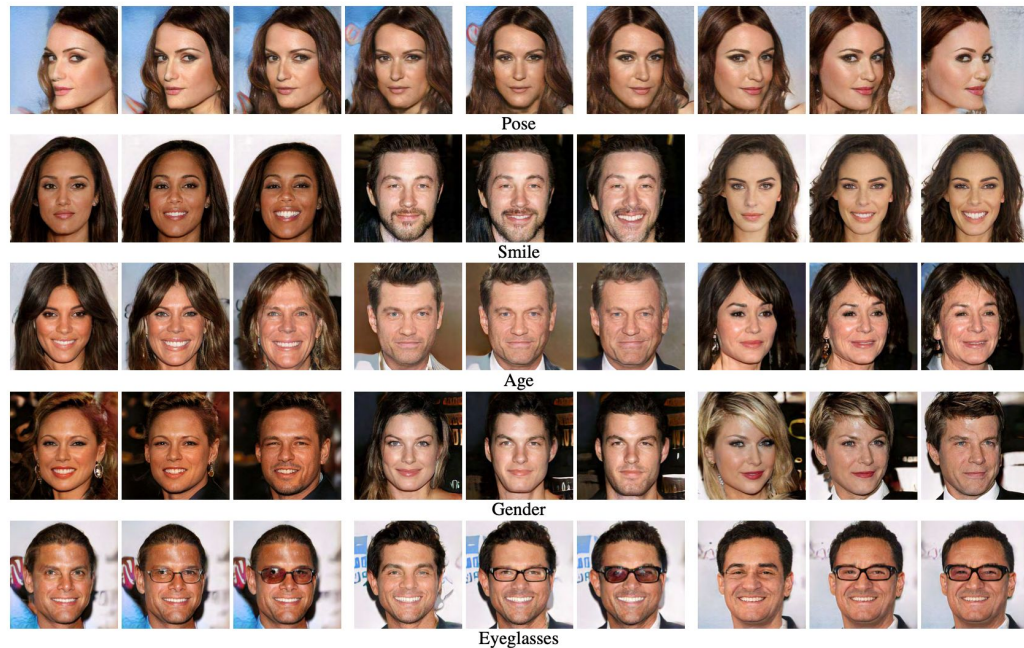


Figure 4: Single attribute manipulation results. The first row shows the same person under gradually changed poses. The following rows correspond to the results of manipulating four different attributes. For each set of three samples in a row, the central one is the original synthesis, while the left and right stand for the results by moving the latent code along negative and positive direction respectively.

Experiment 2: Latent Space Manipulation

Distance Effect of Semantic Subspace

Observe an interesting distance effect that the samples will suffer from severe changes in appearance if being moved too far from the boundary,

Extreme samples are very unlikely to be directly drawn from a standard normal distribution

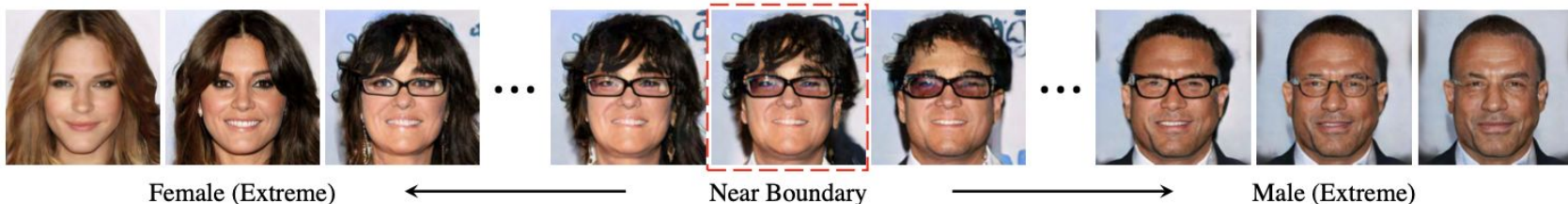
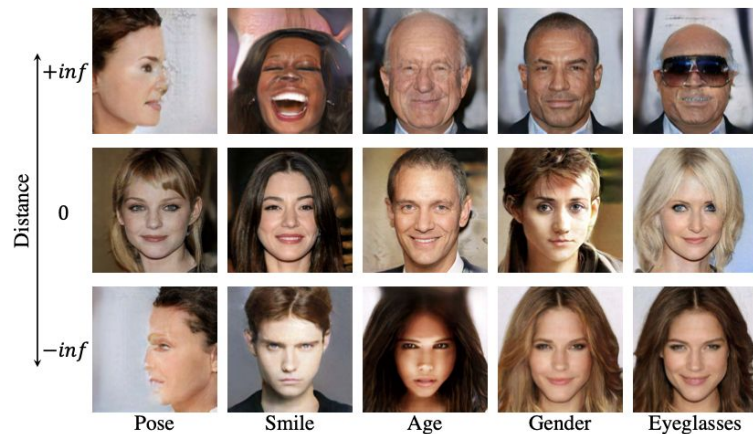


Figure 5: Illustration of the distance effect by taking gender manipulation as an example. The image in the red dashed box stands for the original synthesis. Our approach performs well when the latent code locates close to the boundary. However, when the distance keeps increasing, the synthesized images are no longer like the same person.

Experiment 2: Latent Space Manipulation

Artifacts Correction.

Manually labeled 4K bad synthesis (have artifacts) and then trained a linear SVM to find the separation hyperplane



Figure 6: Examples on fixing the artifacts that GAN has generated. First row shows some bad generation results, while the following two rows present the gradually corrected synthesis by moving the latent codes along the positive “quality” direction.

Experiment 3: Conditional Manipulation

Study the disentanglement between different attributes and evaluate the conditional manipulation approach.

Correlation between Attributes

Cosine similarity of
normal vectors

Correlation coefficient

$$\rho_{A_1 A_2} = \frac{Cov(A_1, A_2)}{\sigma_{A_1} \sigma_{A_2}}$$

Metrics agree, reflects the attribute correlation in the training dataset (male old people are more likely to wear eyeglasses)

Table 2: Correlation matrix of attribute boundaries.

	Pose	Smile	Age	Gender	Eyeglasses
Pose	1.00	-0.04	-0.06	-0.05	-0.04
Smile	-	1.00	0.04	-0.10	-0.05
Age	-	-	1.00	0.49	0.38
Gender	-	-	-	1.00	0.52
Eyeglasses	-	-	-	-	1.00

Table 3: Correlation matrix of synthesized attribute distributions.

	Pose	Smile	Age	Gender	Eyeglasses
Pose	1.00	-0.01	-0.01	-0.02	0.00
Smile	-	1.00	0.02	-0.08	-0.01
Age	-	-	1.00	0.42	0.35
Gender	-	-	-	1.00	0.47
Eyeglasses	-	-	-	-	1.00

Experiment 3: Conditional Manipulation

Effect of conditional manipulation

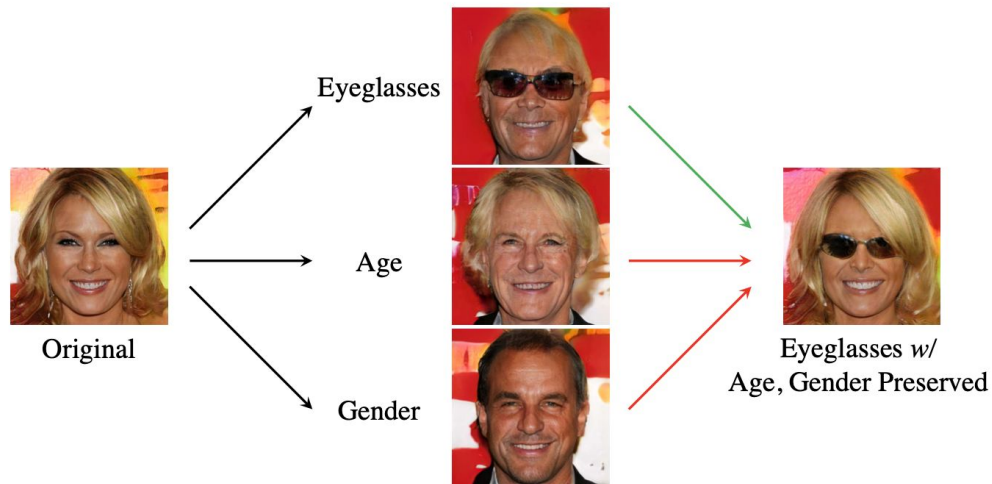


Figure 8: Examples for conditional manipulation with more than one conditions. Left: Original synthesis. Middle: Manipulations along single boundary. Right: Conditional manipulation. **Green** arrow: Primal direction. **Red** arrows: Projection subtraction.

Experiment 3: Conditional Manipulation

Effect of conditional manipulation

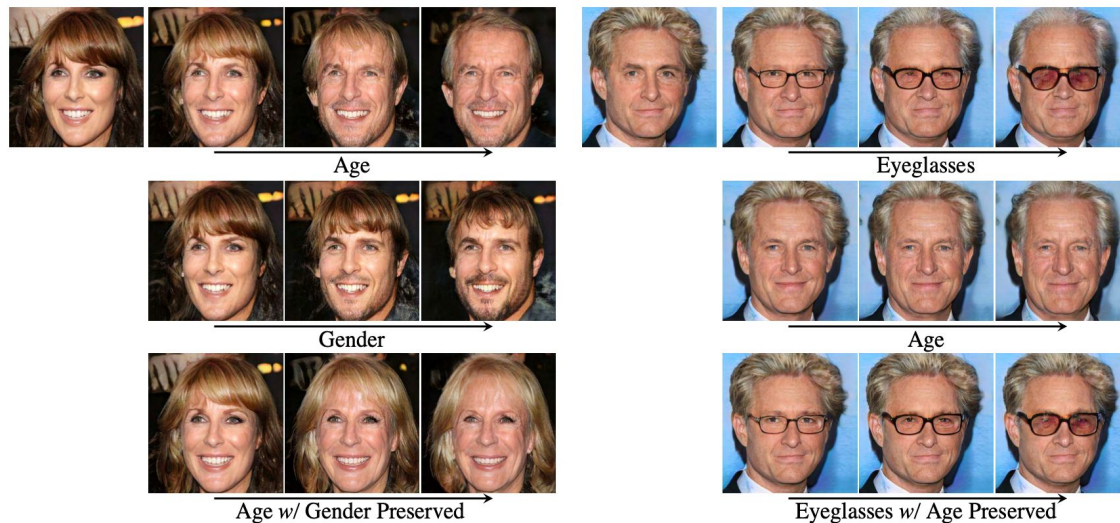


Figure 7: Examples for conditional manipulation. The first two rows show the manipulation results along with the original directions learned by SVMs for two attributes independently. The last row edits the faces by varying one attribute with the other one unchanged.

Experiment 4: Results on StyleGAN

StyleGAN Key Attributes:

PGGan:

Model learns to generate larger images over time

Mapping

Network: Cast random input z as w (“style vector”)

Synthesis

Network: Uses both style vector and noise across layers with AdaIN

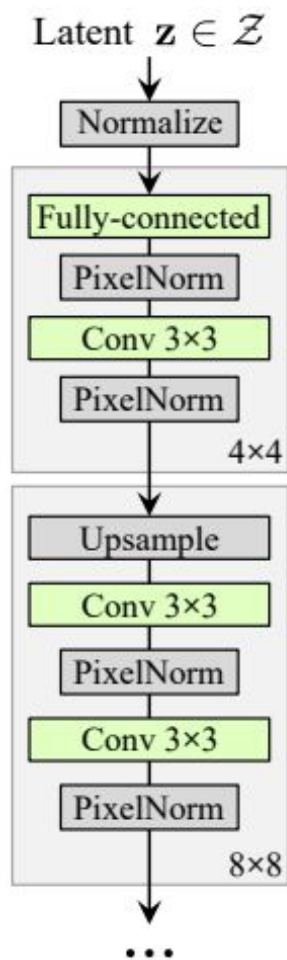
Bilinear

Sampling: Replace nearest neighbor upsampling

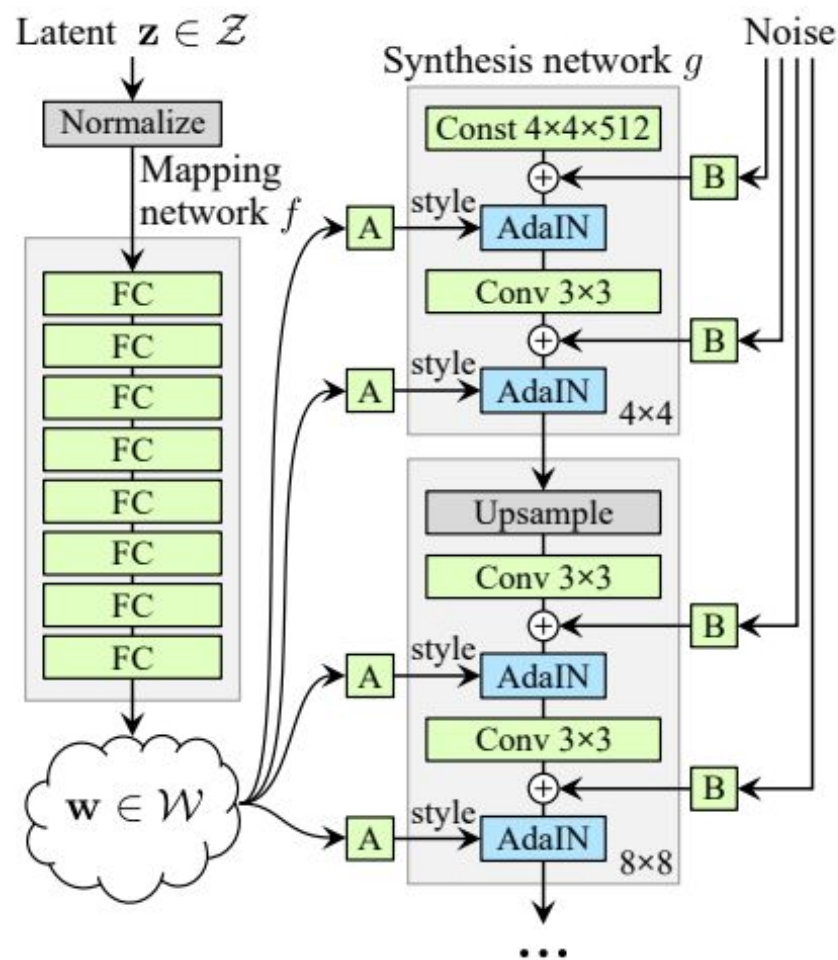
Mixing

Regularization: Use 2 latent codes instead of one to generate

Datasets: trained on CelebA-HQ and FFHQ (new dataset)



(a) Traditional



(b) Style-based generator

Experiment 4: Results on StyleGAN

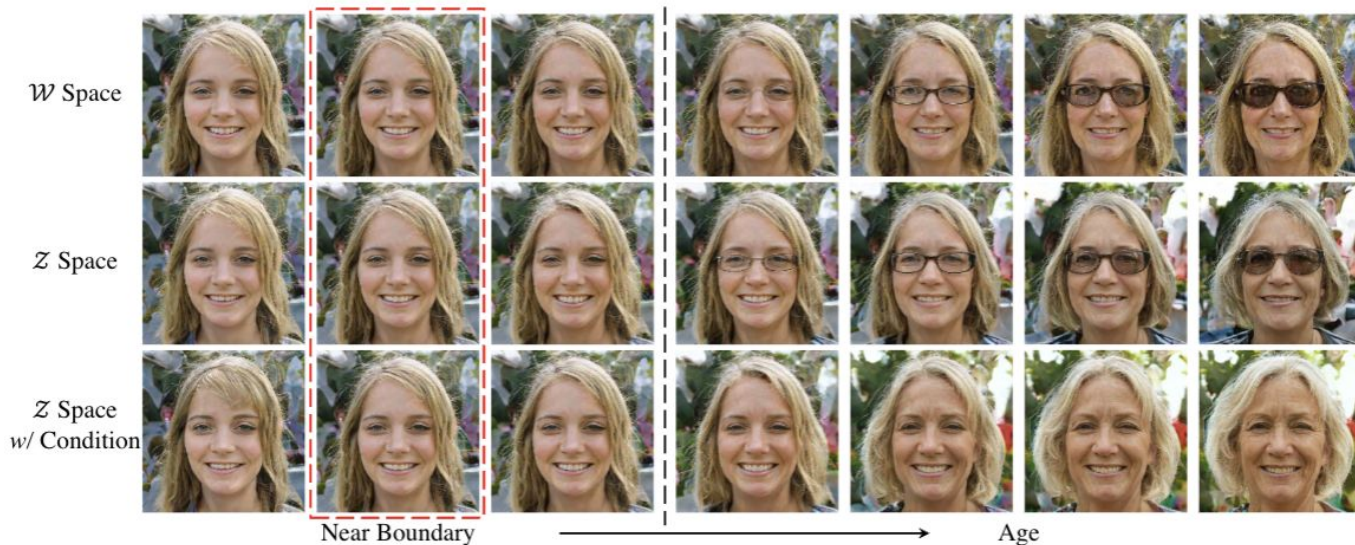
Eyeglasses/age
direction from
StyleGAN

Analysis on Z and W
reveals:

- W learns more
disentangled
representation

- W performs better in
long-distance
manipulation

- W capture attribute
correlations



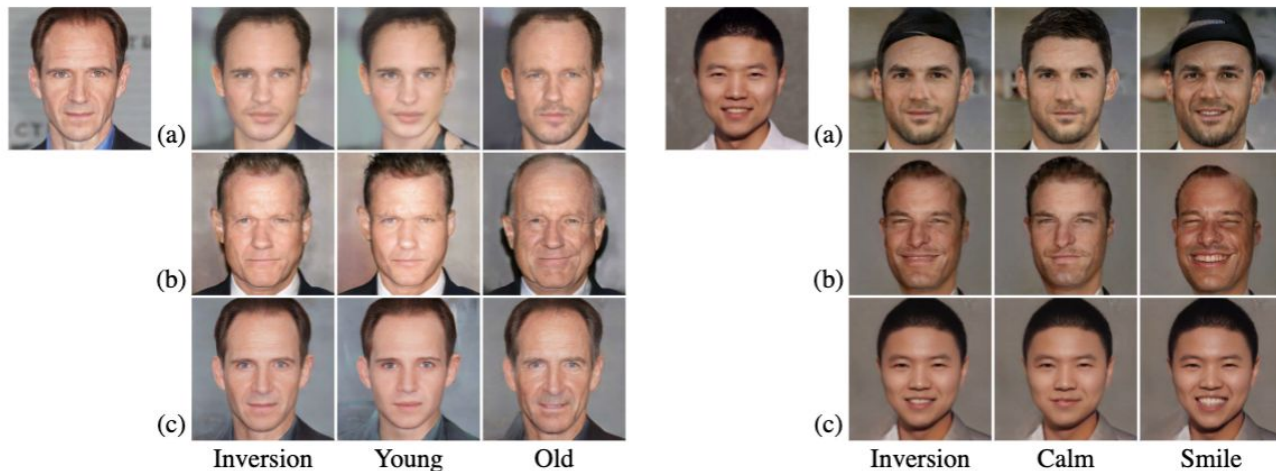
Experiment 5: Real Image Manipulation

-Take a real face image, invert to latent code, and use InterFaceGAN to face edit

Latent Code

Optimization:

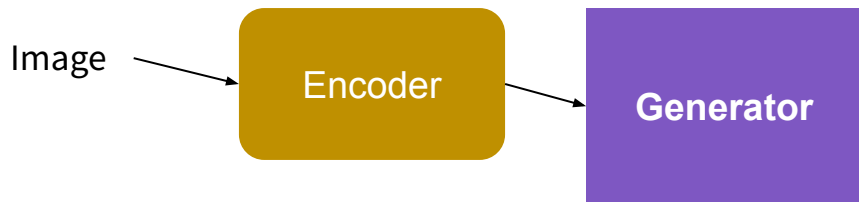
Optimize latent code with fixed generator to minimize pixel-wise reconstruction error



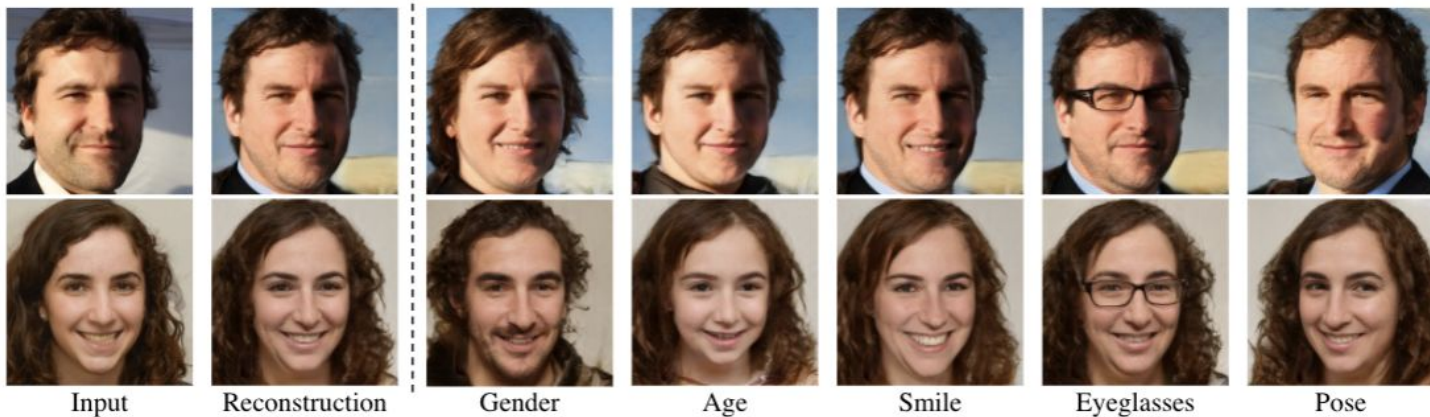
(a) PGGAN with optimization-based inversion method, (b) PGGAN with encoder-based inversion method, (c) StyleGAN with optimization-based inversion method.

Experiment 5: Real Image Manipulation

Encoder-Based: Extra encoder learns inverse mapping from image to latent code by training with generator and discriminator



Testing latent space of LIA



Discussion Questions

- 1) Do you believe the use of GANs for generating photo-realistic images should be regulated? (Potential misuse of such tools)
- 2) Why do we think there are such simple linear boundaries separating binary semantics in the latent space of GANs (is it only for face GANs)?
- 3) How can analyzing semantic representations be used to detect biases in the training data?
- 4) Have you ever used GANs for photo generation, if so how realistic did you find them?
- 5) How relevant is this paper given that GANs are progressively going out of style (diffusion taking over)